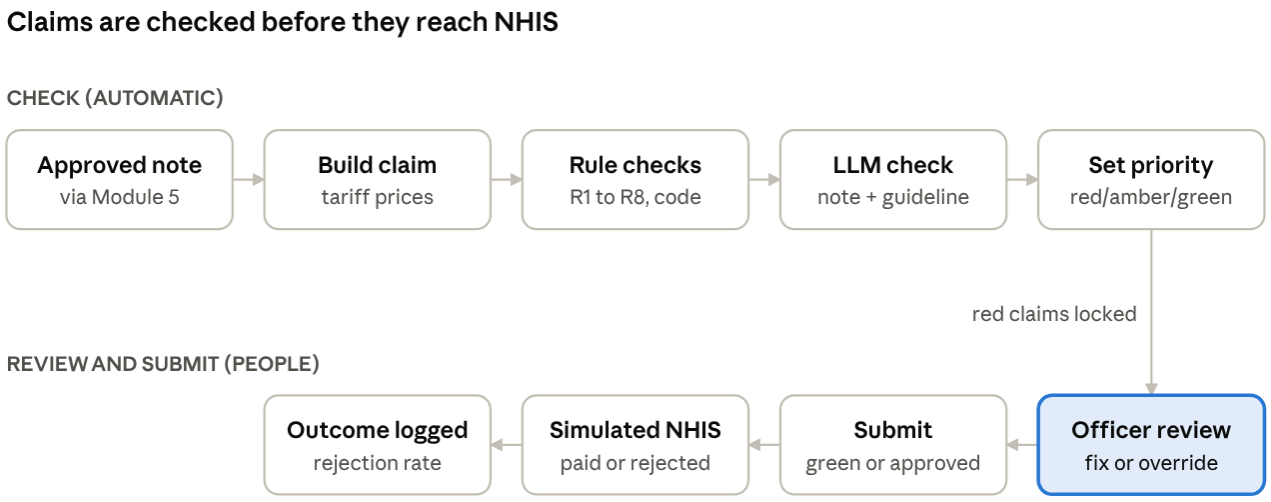
MediFlow AI — Module 6 Architecture (Claims Checker)

Module 6 builds the NHIS claim from the approved note, catches errors with rule checks and an LLM review before submission, and proves the effect by measuring the rejection rate before and after against a simulated NHIS. Each section below can be handed to a coding agent as-is.

Flow at a glance



Module 6 flow · check, review, submit

The top row runs automatically and only raises flags. The bottom row is where a billing officer fixes or overrides them, and only a person submits to NHIS.

Role, concepts and scope

Module 6 protects the clinic's income: it makes sure each NHIS claim is right before it is sent, so it is paid the first time.

Key concepts

- **Claim:** the bill the clinic sends NHIS after treating an NHIS patient: diagnosis, consultation, medicines and their NHIS prices.
- **Rejection:** NHIS refuses to pay, usually for a wrong or missing code, an uncovered drug, a drug that doesn't fit the diagnosis, expired membership, missing fields or wrong prices. The clinic loses the money or waits to resubmit.

- **Rejection rate:** rejected claims ÷ submitted claims. Module 6's headline KPI is this rate before vs after MediFlow.

Connections

- **From Module 5:** the coordinator's `create_claim_draft` tool calls Module 6 (this replaces the placeholder). When the nurse approves that action, the claim goes to the billing officer's queue.
- **Backup trigger:** with `COORDINATOR_ENABLED=false`, note approval calls Module 6 directly.
- **Uses Module 1:** the approved note, and its masking step before the LLM check.
- **Uses Module 2:** the Drug List (coverage, prices, max doses) and the Treatment Guidelines (which drugs fit which diagnosis).

Why there is no machine learning model here: the rule checks and the LLM check already explain every flag. A model trained on synthetic claim history would mostly relearn our own rules, adding training work without adding insight. Claims are prioritised with a simple red / amber / green status instead.

Components

Module 6 runs in two modes: **live** (builds the claim from an approved note) and **check** (checks a claim that already exists, used for the evaluation's 200 test claims). C2 onward is identical in both.

C1. Build the claim (code, live mode)

- From the approved note: patient, visit date, diagnosis, diagnosis_code, medications.
- Add the consultation fee and one line per drug: quantity = dose × frequency × duration_days; unit price from the `tariffs` table; line total = quantity × price.
- Prices always come from the tariff table, never from the LLM.

C2. Rule checks (code)

- Run the rules in *Rule checks*. Each returns pass, warn or block, with a message.

C3. LLM check (reasoning on the note)

- Mask the note with Module 1's masking step.
- Ask Module 2 for the diagnosis's treatment section (`search_guidelines`).
- Send the masked note, the claim lines and the guideline excerpts to Claude with the prompt in *The LLM check prompt*. The answer is forced into JSON.
- The LLM only raises queries; it never changes the claim.

C4. Set priority (code)

Status	Rule	What the officer can do
Red	Any rule returned block	Must fix; Submit is locked
Amber	Any warn, or any LLM query	Fix, or override with a written reason
Green	Everything passed	Submit

C5. Billing officer review

- A claims queue sorted red, amber, green. Opening a claim shows the lines, every flag with its reason and source, the note beside it, and a suggested fix where one exists (e.g. "Use code B54").
- The officer edits lines, applies a suggested fix, or overrides an amber flag with a reason. Edits re-run C2–C4.

C6. Submit to the simulated NHIS

- Submit sends the claim to the simulated NHIS checker (see *The simulated NHIS checker*), which returns paid or rejected with reasons.

C7. Record the outcome

- Save the result. Rejected claims return to the queue with NHIS's reasons, ready to fix and resubmit. The dashboard shows rejection rate and GHS paid.

The claim format

Field	Type	Source	Example
claim_id	text	System	CL-00001
note_id, patient_id	text	Module 1 note	N-0001, MF-00001
nhis_number	text	Patient record	NHIS-48213907
visit_date	date	Note	2026-10-07
diagnosis, diagnosis_code	text	Note	Uncomplicated malaria, B54
lines	list	C1	see below
lines[].item_type	consultation / drug	C1	drug

Field	Type	Source	Example
lines[].item_name	text	Note or tariff table	Artemether-lumefantrine 20/120 mg
lines[].quantity	integer	dose × frequency × days	24
lines[].unit_price_ghs	number	Tariff table	1.25
lines[].line_total_ghs	number	quantity × unit price	30.00
total_ghs	number	Sum of lines	55.00
status	red / amber / green	C4	green
submission_status	draft, ready, submitted, paid, rejected	C5–C7	draft

All prices are fictional and come from `data/tariffs.csv`.

Rule checks

Code checks that need no AI. **Block** stops submission; **warn** needs the officer to confirm.

ID	Check	Result if it fails	Suggested fix shown
R1	NHIS membership active on the visit date	Block	"Membership expired on {date}; bill as cash or ask patient to renew"
R2	diagnosis_code present and in the allowed code list	Block	Closest valid code for the diagnosis text
R3	Every drug is NHIS-covered (Drug List)	Block	"Not covered; remove from claim or bill patient"
R4	Quantity within the Drug List's maximum daily dose × days	Block	Maximum allowed quantity
R5	Every unit price equals the tariff table	Block (check mode)	Correct price from the table

ID	Check	Result if it fails	Suggested fix shown
R6	Required fields filled: nhis_number, visit_date, diagnosis_code, at least one line	Block	Name of the missing field
R7	Line totals and claim total add up	Block	Recalculated totals
R8	Claim older than 60 days after visit_date	Warn	"Submit soon; NHIS may reject late claims"

In live mode, R5 and R7 always pass because C1 builds prices and totals from the table; they matter in check mode, where a claim may have been prepared by hand.

The LLM check prompt

The LLM handles the two checks that need understanding rather than lookups: **L1, does each drug fit the diagnosis?** and **L2, does the note support the diagnosis?** Prompt stored as `prompts/claim_check_v1.txt`:

```
You review NHIS claims for MediFlow Clinic in Ghana before they are
submitted. Use ONLY the clinical note and guideline excerpts below.

Answer two questions using the tool:

L1. For each drug line: is this drug appropriate for the diagnosis
    according to the guideline excerpts?
    status: "ok" or "query"; reason (one sentence); citation (chunk ID).

L2. Does the note contain evidence for the diagnosis: symptoms,
    exam findings or tests that the guideline associates with it?
    status: "ok" or "query"; reason (one sentence, quote the note's
    words if possible); citation (chunk ID).

Rules:
- Raise a query only when the note or guideline gives a clear reason.
  If unsure, return "ok" for L1 and "query" with reason "unclear"
  for L2.
- Do not suggest new drugs, doses or diagnoses.
- Do not comment on prices or NHIS coverage; code checks those.
```

```
Diagnosis: {diagnosis} ({diagnosis_code})
Clinical note (masked): {note_fields}
Claim drug lines: {drug_lines}
Guideline excerpts: {chunks_with_ids}
```

Every query appears as an amber flag with its reason and a link to the cited guideline text.

The simulated NHIS checker

We cannot submit to the real NHIS, so a separate service plays NHIS. It lives in `sim_nhis/`, has its own rules file, and does not import any Module 6 code, so it is a fair, independent judge.

Its rules: a claim is rejected if any of these is true.

NHIS rule	Mirrors
Membership not active on visit date	R1
Code missing or not in the code list	R2
Drug not covered	R3
Quantity above maximum	R4
Price differs from tariff	R5
Required field missing	R6
Drug not on the guideline's list for that diagnosis (lookup table built from the Treatment Guidelines)	L1
Note lacks any symptom or test from the diagnosis's keyword list	L2
Random documentation query on 2% of claims (seeded)	Nothing; real NHIS also rejects some claims for reasons a clinic can't predict

Why the 2% random rejection: without it, the "after" rejection rate could reach 0%, which no reviewer would believe. With it, results look like the real world.

Output: `{claim_id, decision: paid | rejected, reasons: [...], amount_paid_ghs}`.

Guardrails

The rule behind every guardrail: the AI may question a claim, but only code sets prices and only a person submits.

Guardrail	Rule	If it fails
Prices from the table only	Unit prices come from `tariffs`, never from the LLM	Enforced in C1; R5 catches hand edits
LLM queries only	The LLM cannot edit lines, codes or totals	Its output schema has no edit fields
Red locks Submit	Any block rule disables Submit	Button disabled with the reason
Overrides need a reason	Amber flags can be overridden only with a typed reason	Reason stored with officer name
Masking	The note is masked before the LLM sees it	Module 1 masking reused
LLM failure	Schema invalid after one retry, or timeout (20 seconds)	Claim set to amber: "AI check unavailable, review manually"
Grounded queries	Every LLM query cites a guideline chunk	Query without a citation is dropped and logged
Human submits	No claim is sent automatically	Submit is a billing officer action
Full audit	Every check, flag, edit, override and submission is stored	—

Storage, tech and API

Table	Key columns
`claims`	claim_id, note_id, patient_id, nhis_number, visit_date, diagnosis, diagnosis_code, total_ghs, status, submission_status, mode (`live`, `check`), created_at
`claim_lines`	claim_id, item_type, item_name, quantity, unit_price_ghs, line_total_ghs

Table	Key columns
`claim_flags`	claim_id, check_id (R1–R8, L1, L2), result (`warn`, `block`, `query`), message, suggested_fix, citation, overridden_by, override_reason
`nhis_submissions`	claim_id, attempt, decision, reasons, amount_paid_ghs, submitted_at
`tariffs`	item_name, item_type, unit_price_ghs
`nhis_membership`	patient_id, nhis_number, valid_from, valid_to

Tech: Python rule engine (plain functions, one per rule); Claude through the shared provider layer for L1/L2; the simulated NHIS as a separate small FastAPI app; React claims queue and claim screen.

API endpoints

- `POST /claims/draft/{note_id}`: live mode; build and check (called by Module 5's tool).
- `POST /claims/check`: check mode; check a pre-built claim.
- `GET /claims/queue`: claims sorted red, amber, green.
- `PUT /claims/{id}`: officer edits (re-runs checks).
- `POST /claims/{id}/override` with flag_id and reason.
- `POST /claims/{id}/submit`: send to the simulated NHIS.
- Simulated NHIS: `POST /sim-nhis/adjudicate`.

KPIs and evaluation

The headline result comes from one experiment on 200 test claims (140 clean, 60 with one planted error each). Run with `python evals/run_module6.py`.

The experiment

1. **Before MediFlow:** send all 200 claims straight to the simulated NHIS. Record the rejection rate.
2. **With MediFlow:** run all 200 through Module 6 in check mode. Claims where a flag correctly identifies the planted error are treated as fixed using the answer key's correction; claims with missed errors go through unchanged.
3. Send the result to the simulated NHIS. Record the new rejection rate.

KPI	How it is measured	Target
Rejection rate before	Rejected ÷ 200, no checks	About 30% (by design)
Rejection rate after	Rejected ÷ 200, after Module 6	⩽ 8%
Error catch rate	Planted errors flagged ÷ 60	⩾ 90%
Catch rate, LLM checks only	L1/L2 errors flagged ÷ 17	⩾ 80%
False alarms	Clean claims flagged ÷ 140	⩽ 10%
GHS recovered	Value of claims rejected before but paid after	Reported
Time per claim check	Rule checks + LLM check	< 15 seconds

A results table per error type (caught / missed) is written to `docs/evals.md`, so you can say exactly which errors the system catches and which it doesn't.

Definition of done

Module 6 is done when all of these work:

- [ ] Approving the claim action in Module 5's care plan creates a claim in the billing queue (the placeholder is gone)
- [ ] The claim is built from the note with prices from the tariff table
- [ ] Each rule R1–R8 and LLM check L1–L2 produces its flag on a test claim that breaks it
- [ ] Red claims cannot be submitted; amber claims need a fix or a written override
- [ ] Submitting returns paid or rejected from the simulated NHIS, and rejected claims return to the queue with reasons
- [ ] The evaluation script prints before and after rejection rates, catch rates per error type, false alarms and GHS recovered
- [ ] The dashboard shows rejection rate and GHS paid

Synthetic data prompt

No training data is needed, only reference tables and the 200 test claims. Run this after Module 2's documents exist. Give it to your coding agent:

```
Write data/generators/generate_claims.py (seed 42, numpy/pandas, no
LLM needed). Read data/knowledge_base/drug_list.md and
treatment_guidelines.md first.

Output 1: data/tariffs.csv
  One row for the consultation fee (fictional, e.g. GHS 25.00) and one
  row per drug in the Drug List with its NHIS tariff (unit price).

Output 2: data/nhis_membership.csv
  For every patient (MF-00001 onward): nhis_number, valid_from,
  valid_to. About 10% expired before 2026.

Output 3: data/sim_nhis/drugs_by_diagnosis.json and
  diagnosis_keywords.json
  From the Treatment Guidelines: which drugs are allowed for each
  diagnosis, and 4-6 symptom/test keywords per diagnosis.

Output 4: evals/module6_claims.json, 200 claims in the claim format
  (claim_id, patient_id, nhis_number, visit_date, diagnosis,
  diagnosis_code, lines with quantity, unit_price_ghs, line_total_ghs,
  total_ghs) plus a short note per claim (chief_complaint, symptoms,
  exam_findings, diagnosis, medications), spread across the 10
  diagnoses.

  140 clean claims. 60 claims with exactly ONE planted error:
    R1 expired membership: 9      R2 wrong or missing code: 9
    R3 uncovered drug: 8          R4 quantity above maximum: 6
    R5 wrong unit price: 6        R6 missing required field: 5
    L1 drug that doesn't fit the diagnosis: 10
    L2 note with no supporting symptom or test: 7

  Each claim also has: planted_error (null or the check ID),
  correction (the fixed value or line, for errors).

Output 5: print counts per error type and the total claim value.
```

When it's done, open 3 claims yourself: one L1 (does the drug really not fit?), one L2 (is the note really missing evidence?), and one clean claim (is it really clean?).